WIKIPEDIA 25
25 years of the free encyclopedia

Fibonacci search technique

In computer science, the **Fibonacci search technique** is a method of searching a sorted array using a divide and conquer algorithm that narrows down possible locations with the aid of Fibonacci numbers.[1] The technique is conceptually similar to a binary search, which repeatedly splits the search interval into two equal halves. Fibonacci search, however, splits the array into two unequal parts, with sizes that are consecutive Fibonacci numbers.

This method has a key advantage on older computer hardware where arithmetic division or bit-shifting operations were computationally expensive compared to addition and subtraction. Since the Fibonacci sequence is based on addition, this search method could be implemented more efficiently. While on modern CPUs the performance difference is often negligible, the algorithm remains of theoretical and historical interest. On average, Fibonacci search performs about 4% more comparisons than binary search,[2] and it has an average-case complexity and worst-case complexity of $O(\log n)$ (see Big O notation).

Fibonacci search can also have an advantage in searching data stored in certain structures, such as on a magnetic tape, where seek times are heavily dependent on the distance from the current head position. It is derived from Golden section search, an algorithm by Jack Kiefer (1953) to search for the maximum or minimum of a unimodal function in an interval.[3]

Algorithm

The Fibonacci search technique works by first finding the smallest Fibonacci number that is greater than or equal to the length of the array. Let this Fibonacci number be F_m. The algorithm then uses the preceding Fibonacci numbers, F_{m-1} and F_{m-2}, to probe the array at an index, narrowing the search space in each step. The size of the step decreases at each iteration according to the Fibonacci sequence.

Let k be defined as an element in F, the array of Fibonacci numbers. $n = F_m$ is the array size. If n is not a Fibonacci number, let F_m be the smallest number in F that is greater than n.

The array of Fibonacci numbers is defined where $F_{k+2} = F_{k+1} + F_k$, when $k \geq 0$, $F_1 = 1$, and $F_0 = 1$.

To test whether an item is in the list of ordered numbers, follow these steps:

1. Set $k = m$.
2. If $k = 0$, stop. There is no match; the item is not in the array.
3. Compare the item against element in F_{k-1}.
4. If the item matches, stop.

5. If the item is less than entry F_{k-1}, discard the elements from positions $F_{k-1} + 1$ to n. Set $k = k - 1$ and return to step 2.
6. If the item is greater than entry F_{k-1}, discard the elements from positions 1 to F_{k-1}. Renumber the remaining elements from 1 to F_{k-2}, set $k = k - 2$, and return to step 2.

Alternative implementation (from "Sorting and Searching" by Knuth[4]):

- Given a table of records R_1, R_2, ..., R_N whose keys are in increasing order $K_1 < K_2 < \ldots < K_N$, the algorithm searches for a given argument K. Assume $N+1 = F_{k+1}$

1. [Initialize] $i \leftarrow F_k$, $p \leftarrow F_{k-1}$, $q \leftarrow F_{k-2}$ (throughout the algorithm, p and q will be consecutive Fibonacci numbers)
2. [Compare] If $K < K_i$, go to *Step 3*; if $K > K_i$ go to *Step 4*; and if $K = K_i$, the algorithm terminates successfully.
3. [Decrease i] If $q=0$, the algorithm terminates unsuccessfully. Otherwise set $(i, p, q) \leftarrow (i - q, q, p - q)$ (which moves p and q one position back in the Fibonacci sequence); then return to *Step 2*
4. [Increase i] If $p=1$, the algorithm terminates unsuccessfully. Otherwise set $(i, p, q) \leftarrow (i + q, p - q, 2q - p)$ (which moves p and q two positions back in the Fibonacci sequence); and return to *Step 2*

The two variants of the algorithm presented above always divide the current interval into a larger and a smaller subinterval. The original algorithm,[1] however, would divide the new interval into a smaller and a larger subinterval in Step 4. This has the advantage that the new i is closer to the old i and is more suitable for accelerating searching on magnetic tape.

Example

Suppose we want to search for the number `x = 85` in the following sorted array of `n = 11` elements:

```
[10, 22, 35, 40, 45, 50, 80, 82, 85, 90, 100]
```

1. **Find the Fibonacci numbers:**
   - The smallest Fibonacci number greater than or equal to `n = 11` is $F_7 = 13$. So, `m = 7`.
   - The two preceding Fibonacci numbers are $F_5 = 5$ and $F_6 = 8$.
   - We will maintain an offset for the subarray we are searching. Initially, `offset = -1`.
2. **First comparison:**
   - The first comparison index is `i = min(offset + F_6, n-1) = min(-1 + 8, 10) = 7`.
   - We compare `x` with the element at index 7: `array[7]` is 82.
   - Since `x = 85` is greater than 82, we eliminate the elements from the start of the array up to index 7. The new search space is the subarray from index 8 onwards.

- We update the Fibonacci numbers by moving two steps down: `m = m - 2 = 6` (so the new Fibonacci numbers for the step size will be $F_5 = 5$ and $F_4 = 3$).
- We update the offset: `offset = i = 7`.

3. **Second comparison:**
   - The next comparison index is `i = min(offset + F_4, n-1) = min(7 + 3, 10) = 10`.
   - We compare x with the element at index 10: `array[10]` is 100.
   - Since `x = 85` is less than 100, we eliminate the elements from index 10 to the end.
   - We update the Fibonacci numbers by moving one step down: `m = m - 1 = 5` (new step sizes will be $F_4 = 3$ and $F_3 = 2$).
   - The offset remains `offset = 7`.
4. **Third comparison:**
   - The next comparison index is `i = min(offset + F_3, n-1) = min(7 + 2, 10) = 9`.
   - We compare x with the element at index 9: `array[9]` is 90.
   - Since `x = 85` is less than 90, we eliminate the elements from index 9 to the end.
   - We update the Fibonacci numbers by moving one step down: `m = m - 1 = 4` (new step sizes will be $F_3 = 2$ and $F_2 = 1$).
   - The offset remains `offset = 7`.
5. **Fourth comparison:**
   - The next comparison index is `i = min(offset + F_2, n-1) = min(7 + 1, 10) = 8`.
   - We compare x with the element at index 8: `array[8]` is 85.
   - The item is found at index 8. The algorithm terminates successfully.

See also

- Search algorithms

References

1. Ferguson, David E. (1960). "Fibonaccian searching" (https://doi.org/10.1145%2F367487.367496). *Communications of the ACM*. **3** (12): 648. doi:10.1145/367487.367496 (https://doi.org/10.1145%2F367487.367496). S2CID 7982182 (https://api.semanticscholar.org/CorpusID:7982182). Note that the running time analysis is this article is flawed, as pointed out by Overholt in 1972 (published 1973).
2. Overholt, K. J. (1973). "Efficiency of the Fibonacci search method". *BIT Numerical Mathematics*. **13** (1): 92–96. doi:10.1007/BF01933527 (https://doi.org/10.1007%2FBF0193

3527). S2CID 120681132 (https://api.semanticscholar.org/CorpusID:120681132).

3. Kiefer, J. (1953). "Sequential minimax search for a maximum" (https://doi.org/10.1090%2FS0002-9939-1953-0055639-3). *Proceedings of the American Mathematical Society*. **4** (3): 502–506. doi:10.1090/S0002-9939-1953-0055639-3 (https://doi.org/10.1090%2FS0002-9939-1953-0055639-3).
4. Knuth, Donald E. (2003). *The Art of Computer Programming*. Vol. 3 (Second ed.). p. 418.

- Lourakis, Manolis. "Fibonaccian search in C" (http://www.ics.forth.gr/~lourakis/fibsrch/). Retrieved January 18, 2007. Implements the above algorithm (not Ferguson's original one).

Retrieved from "https://en.wikipedia.org/w/index.php?title=Fibonacci_search_technique&oldid=1351697302"